

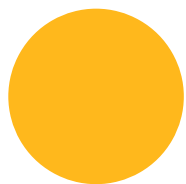
Segmento *DIGITAL*: **MOBILE**

IMPACTO
por **BCP**



¿Quién escribió este código?

*Yo, hace seis
meses.
¿Te ha pasado? No
estás solo.*



"Un momento que me hizo reflexionar..."

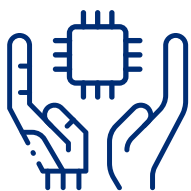
Tuve que entender un código complejo para una tarea urgente. Me frustré. Lo había escrito yo mismo, meses atrás. Desde entonces, empecé a aplicar principios de Clean Code y mi vida cambió."



¿Qué es Clean Code?

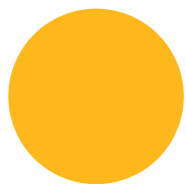
Significa escribir código que sea fácil de leer, entender y mantener.

Un buen código no necesita comentarios para explicar qué hace. Se explica por sí solo con nombres claros, funciones pequeñas y una estructura simple.



¿Cómo lo aplicaremos hoy?

A lo largo de esta presentación, aplicaremos principios de Clean Code usando el lenguaje de Kotlin



1

Nombres Claros y Significativos



Evitemos:

```
4  
5   val e = "Juan Pérez"  
6
```



Hagamos:

```
7  
8   val employeeFullName = "Juan Pérez"  
9
```

*Usa nombres que expliquen la intención
sin necesidad de comentarios.*

2

Funciones Cortas y con una sola responsabilidad



Evitemos:

```
11  fun process() {  
12      // muchas líneas  
13  }  
14
```



Hagamos:

```
16  fun process() {  
17      validate()  
18      fetchData()  
19      updateUI()  
20  }
```

Divide funciones grandes en otras más pequeñas con un solo propósito

3

Evita comentarios innecesarios



Evitemos:

```
31 // Esta es la función para calcular el IMC
32 fun calcularIMC(w: Double, h: Double): Double {
33 // Multiplicamos la altura por sí misma
34     val z = h * h
35 // Dividimos el peso entre la altura al cuadrado
36     return w / z
37 }
```



Hagamos:

```
39 fun calculateBMI(weight: Double, height: Double): Double {
40     return weight / (height * height)
41 }
```

El buen código se explica por sí solo

4

Evita if/else anidados (y anidaciones profundas)



Evitemos:

```
63  if (user != null) {  
64      if (user.isActive) {  
65          if (user.role == "Admin") {  
66              sendReport(user)  
67          }  
68      }  
69  }
```



Hagamos:

```
76      if (user == null) return  
77      if (!user.isActive) return  
78      if (user.role != ADMIN) return  
79      sendReport(user)
```

*Menos niveles de anidamiento
= más claridad mental.*

5

Evita números mágicos



Evitemos:

```
85     if (speed > 120) {  
86         // penalización  
87     }
```



Hagamos:

```
92     if (speed > MAX_SPEED) {  
93         // penalización  
94     }
```

Los números con nombre explican su propósito

6

Eliminar código muerto o no usado



Evitemos:

```
100 //código antiguo comentado que ya no se usa
101 ✓ fun calculateDiscount(price: Double): Double {
102     // val discount = price * 0.2 // descuento fijo (obsoleto)
103     // return price - discount
104
105     if (price > 100) {
106         return price * 0.9
107     }
108
109     return price
110 }
```



Hagamos:

```
113 fun calculateDiscount(price: Double): Double {
114     if (price > 100) {
115         return price * 0.9
116     }
117     return price
118 }
```

El código comentado no es documentación. Si ya no lo necesita, elimínalo



***Recordar es volver a vivir.. y en
el mundo del código, también es
reaprender.***

***Siempre hay una nueva
oportunidad para escribir mejor.***

***Mantén tu código limpio, tu mente
también.***

***Porque el mejor código es el que
no necesita ser explicado.***

